

TestPlatform

Rapport d'Execution de Campagne

ms_gestion

Projet: ms_gestion

Version: 2.5

Environnement: ms_gestion_env

Date: 05/06/2026 09:43

ECHEC

Resume executif

Total	Succes	Echecs	Erreurs	Duree
3	0	3	0	244.6 s

Taux de reussite



Evolution vs campagne precedente: N/A

Premiere campagne

Informations du contexte

Projet	ms_gestion
Description	test

Suites

Nom	Type	Depot Git	Branche
Suite UNIT	UNIT	N/A	main

Cas de test

Titre	Type	Priorite	Risque	Script
test case add	UNIT	3	CRITICAL	IA
test add () suite test	UNIT	1	CRITICAL	IA
project create	UNIT	1	HIGH	IA

Environnement

URL Web	N/A
URL API	N/A
Repository	https://github.com/ilyeschaabani/Automatisation_intelligente_des_tests_applicatifs
Branche	main
Base de donnees	POSTGRESQL

Resultats details

test case add

FAILURE

Type	UNIT	Duree	9.4 s
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 2.024 s <<< FAILURE! -- in suites.unit.TestCaseServiceTest

test add () suite test

FAILURE

Type	UNIT	Duree	8.3 s
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.542 s <<< FAILURE! -- in suites.unit.TestSuiteServiceUnitTest

project create

FAILURE

Type	UNIT	Duree	7.5 s
Script	IA	Erreur	[COMPILATION_ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.459 s <<< FAILURE! -- in suites.unit.ProjectServiceTest

Analyse IA

test case add

Erreurs extraites	<pre>[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 2.024 s <<< FAILURE! -- in suites.unit.TestCaseServiceTest [ERROR] suites.unit.TestCaseServiceTest.testAddTestCase -- Time elapsed: 0.058 s <<< FAILURE! [ERROR] Failures: [ERROR] TestCaseServiceTest.testAddTestCase:101 [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire- plugin:3.2.5:test (default-test) on project ms_gestion: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec- 17909927170454117453\Backend\ms_gestion\target\surefire-reports for the individual test results. [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	Cause : L'erreur de compilation semble provenir du fait que la classe `Project` n'est pas correctement importée, elle est probablement absente dans le package Java actuel. Cela pourrait causer une erreur au moment de l'exécution car Mockito ne sait pas quoi faire avec cette référence. Correction : Assurez-vous que la classe `Project` est correctement importée et définie dans votre projet. Si elle n'est pas présente, vous devrez peut-être l'ajouter à vos sources ou mettre à jour les chemins d'accès de vos dépendances. Conseil : Assurez-vous que toutes les classes et packages utilisés dans votre script sont correctement importés, avec la bonne syntaxe et le chemin approprié. Mockito nécessite une connaissance précise des classes qu'il va devoir intercepter ou mockéer pour fonctionner correctement.

test add () suite test

Erreurs extraites	<pre>[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.542 s <<< FAILURE! -- in suites.unit.TestSuiteServiceUnitTest [ERROR] suites.unit.TestSuiteServiceUnitTest.test_add_happyPath -- Time elapsed: 0.032 s <<< FAILURE! java.lang.AssertionError: expected [TestProject] but found [null] [ERROR] Failures: [ERROR] TestSuiteServiceUnitTest.test_add_happyPath:89 expected [TestProject] but found [null] [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire- plugin:3.2.5:test (default-test) on project ms_gestion: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec- 11683413072518821367\Backend\ms_gestion\target\surefire-reports for the individual test results.</pre>
----------------------	---

	<pre>[ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
Analyse IA	Cause : Le test est rencontrant une erreur de compilation car Mockito n'a pas pu initialiser correctement les objets mocks. Lors de l'exécution du script, cela signifie que `testSuiteService` n'est pas instancié correctement avec les dépendances des mocks. Correction : La cause racine de l'erreur est la manière dont Mockito est initialisé dans ce test. Il faut utiliser un bloc `try-with-resources` pour garantir que tous les objets mocks sont correctement fermés après chaque test. Dans le script, veuillez ajouter le code suivant avant l'exécution des tests : <pre>```java @BeforeMethod public void setUp() { try (MockedStatic<SecurityUtils> mocked = Mockito.mockStatic(SecurityUtils.class)) { mocks = MockitoAnnotations.openMocks(this); // ... } } ```</pre> Ce bloc `try-with-resources` va garantir que les objets mocks sont correctement fermés après chaque test, ce qui peut résoudre l'erreur de compilation. Conseil : Pour éviter cet type d'échec, assurez-vous que toutes vos dépendances mocks sont correctement initialisées à chaque test avec Mockito et réglez votre code de manière à ce que les objets mocks soient fermés après chaque test.

project create

Erreurs extraites	<pre>[ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 1.459 s <<< FAILURE! -- in suites.unit.ProjectServiceTest [ERROR] suites.unit.ProjectServiceTest.test_create_happy_path -- Time elapsed: 0.032 s <<< FAILURE! [ERROR] Failures: [ERROR] ProjectServiceTest.test_create_happy_path:44 Runtime Utilisateur non authentifi [ERROR] Tests run: 1, Failures: 1, Errors: 0, Skipped: 0 [INFO] BUILD FAILURE [ERROR] Failed to execute goal org.apache.maven.plugins:maven-surefire- plugin:3.2.5:test (default-test) on project ms_gestion: There are test failures. [ERROR] [ERROR] Please refer to C:\tmp\ms-execution\exec- 17909927170454117453\Backend\ms_gestion\target\surefire-reports for the individual test results. [ERROR] Please refer to dump files (if any exist) [date].dump, [date]-jvmRun[N].dump and [date].dumpstream. [ERROR] -> [Help 1] [ERROR] [ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch. [ERROR] Re-run Maven using the -X switch to enable full debug logging. [ERROR] [ERROR] For more information about the errors and possible solutions, please read the following articles: [ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException</pre>
----------------------	---

Analyse IA	Cause : Le test n'est pas compilé car il manque la méthode `assertEquality` dans le bloc de réponse. Cette méthode n'existe pas, elle doit être remplacée par la méthode `assertEquals` ou `assertTrue`, selon ce que vous essayez de tester. Correction : Remplacez le code suivant: <pre>```java assertEqua ```</pre> Par ceci (assurez-vous d'avoir importé correctement les classes nécessaires et adaptée la comparaison à votre besoin): <pre>```java assertEquals(/*valeur attendue*/, /*valeur réelle*/); // ou assertTrue(/*condition à vérifier*/); ```</pre> Conseil : Assurez-vous que les classes utilisées dans le script de test (com.pfe.platform.ms_gestion.service.ProjectService, com.pfe.platform.ms_gestion.repository.ProjectRepository, etc.) sont bien importées et disponibles pour la compilation du script.
-----------------------	--

Evaluation UX

Aucun test UX execute pour cette campagne

Metriques

Top 5 des tests les plus lents

Test	Suite	Duree
test case add	Suite UNIT	9.4 s
test add () suite test	Suite UNIT	8.3 s
project create	Suite UNIT	7.5 s

Temps d'execution par type

Type	Duree moyenne
UNIT	8.4 s

Tests instables

Aucun test instable detecte

Recommandations

- test case add:

Cause : L'erreur de compilation semble provenir du fait que la classe `Project` n'est pas correctement importée, elle est probablement absente dans le package Java actuel. Cela pourrait causer une erreur au moment de l'exécution car Mockito ne sait pas quoi faire avec cette référence.

Correction : Assurez-vous que la classe `Project` est correctement importée et définie dans votre projet. Si elle n'est pas présente, vous devrez peut-être l'ajouter à vos sources ou mettre à jour les chemins d'accès de vos dépendances.

Conseil : Assurez-vous que toutes les classes et packages utilisés dans votre script sont correctement importés, avec la bonne syntaxe et le chemin approprié. Mockito nécessite une connaissance précise des classes qu'il va devoir intercepter ou mockéer pour fonctionner correctement.

- test add () suite test:

Cause : Le test est rencontrant une erreur de compilation car Mockito n'a pas pu initialiser correctement les objets mocks. Lors de l'exécution du script, cela signifie que `testSuiteService` n'est pas instancié correctement avec les dépendances des mocks.

Correction : La cause racine de l'erreur est la manière dont Mockito est initialisé dans ce test. Il faut utiliser un bloc `try-with-resources` pour garantir que tous les objets mocks sont correctement fermés après chaque test. Dans le script, veuillez ajouter le code suivant avant l'exécution des tests :

```
```java
@BeforeMethod
public void setUp() {
 try (MockedStatic<SecurityUtils> mocked = Mockito.mockStatic(SecurityUtils.class)) {
 mocks = MockitoAnnotations.openMocks(this);
 // ...
 }
}
```
```

Ce bloc `try-with-resources` va garantir que les objets mocks sont correctement fermés après chaque test, ce qui peut résoudre l'erreur de compilation.

Conseil : Pour éviter cet type d'échec, assurez-vous que toutes vos dépendances mocks sont correctement initialisées à chaque test avec Mockito et réglez votre code de manière à ce que les objets mocks soient fermés après chaque test.

- project create:

Cause : Le test n'est pas compilé car il manque la méthode `assertEquality` dans le bloc de réponse. Cette méthode n'existe pas, elle doit être remplacée par la méthode `assertEquals` ou `assertTrue`, selon ce que vous essayez de tester.

Correction : Remplacez le code suivant:

```
```java
assertEqua
```
```

Par ceci (assurez-vous d'avoir importé correctement les classes nécessaires et adaptée la comparaison à votre besoin):

```
```java
assertEquals(/*valeur attendue*/, /*valeur réelle*/);
// ou
assertTrue(/*condition à vérifier*/);
```
```

Conseil : Assurez-vous que les classes utilisées dans le script de test (com.pfe.platform.ms_gestion.service.ProjectService, com.pfe.platform.ms_gestion.repository.ProjectRepository, etc.) sont bien importées et disponibles pour la compilation du script.

Annexe - Scripts generes par IA

test case add

```
package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestCaseRequest;
import com.pfe.platform.ms_gestion.dto.response.TestCaseResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestCase;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.TestCaseRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.LlmService;
import com.pfe.platform.ms_gestion.service.TestCaseService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;
import java.util.Optional;

public class TestCaseServiceTest {

    @Mock private TestCaseRepository testCaseRepository;
    @Mock private TestSuiteRepository testSuiteRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @Mock private LlmService llmService;
    @InjectMocks private TestCaseService testCaseService;

    private AutoCloseable mocks;
    private MockedStatic<SecurityUtils> mockedSecurity;

    @BeforeMethod
    public void setUp() {
        mocks = MockitoAnnotations.openMocks(this);
        mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
        mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mockedSecurity.close();
        mocks.close();
    }

    @Test
    public void testAddTestCase() {
        Long suiteId = 1L;

        Project project = new Project();
        project.setId(10L);
        project.setName("Test Project");

        ProjectMember member = new ProjectMember();
        member.setRole(ProjectMember.Role.ADMIN);
        member.setUserId(1L);
        member.setProject(project);

        TestSuite suite = new TestSuite();
```

```

suite.setId(suiteId);
suite.setType(TestSuite.TestType.UNIT);
suite.setProject(project);

CreateTestCaseRequest request = new CreateTestCaseRequest();
request.setTitle("Login test");
request.setType("UNIT");
request.setRiskLevel("LOW");
request.setPriority(1);
request.setDescriptionAI("Test the add method");

Mockito.when(testSuiteRepository.findById(suiteId))
    .thenReturn(Optional.of(suite));
Mockito.when(projectMemberRepository.findByIdAndUserId(
Mockito.any(), Mockito.eq(1L)))
    .thenReturn(Optional.of(member));

Mockito.when(llmService.generateTestCode(
Mockito.any(), Mockito.any(), Mockito.any()))
    .thenReturn("public class FakeTest {}");

Mockito.when(testCaseRepository.save(Mockito.any()))
    .thenAnswer(inv -> {
TestCase tc = inv.getArgument(0);
tc.setSuite(suite);
return tc;
});

TestCaseResponse response = testCaseService.add(suiteId, request);

Assert.assertNotNull(response);
Assert.assertEquals(response.getTitle(), "Login test");
Assert.assertEquals(response.getRiskLevel(), "LOW");
Mockito.verify(testCaseRepository, Mockito.times(1)).save(Mockito.any());
// verify
Mockito.verify(llmService, Mockito.times(1))
    .generateTestCode(Mockito.any(), Mockito.any(), Mockito.any());
}
}

```

test add () suite test

```

package suites.unit;

import com.pfe.platform.ms_gestion.dto.request.CreateTestSuiteRequest;
import com.pfe.platform.ms_gestion.dto.response.TestSuiteResponse;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.entity.ProjectMember;
import com.pfe.platform.ms_gestion.entity.TestSuite;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.repository.ProjectRepository;
import com.pfe.platform.ms_gestion.repository.TestSuiteRepository;
import com.pfe.platform.ms_gestion.security.SecurityUtils;
import com.pfe.platform.ms_gestion.service.TestSuiteService;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockedStatic;
import org.mockito.Mockito;
import org.mockito.MockitoAnnotations;
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Test;

import java.util.Optional;

public class TestSuiteServiceUnitTest {

```

```
@Mock private TestSuiteRepository testSuiteRepository;
@Mock private ProjectRepository projectRepository;
@Mock private ProjectMemberRepository projectMemberRepository;

@InjectMocks
private TestSuiteService testSuiteService; // pas de new()

private AutoCloseable mocks;
private MockedStatic<SecurityUtils> mockedSecurity;

@BeforeMethod
public void setUp() {
    mocks = MockitoAnnotations.openMocks(this);
    mockedSecurity = Mockito.mockStatic(SecurityUtils.class);
    mockedSecurity.when(SecurityUtils::getCurrentUserId).thenReturn(1L);
}

@AfterMethod
public void tearDown() throws Exception {
    mockedSecurity.close();
    mocks.close();
}

@Test
public void test_add_happyPath() {
    // Arrange
    Long projectId = 1L;

    Project project = new Project();
    project.setId(projectId);

    ProjectMember member = new ProjectMember();
    member.setUserId(1L);
    member.setRole(ProjectMember.Role.ADMIN);
    member.setProject(project);

    TestSuite saved = new TestSuite();
    saved.setId(10L);
    saved.setProject(project);

    // Mock repositories
    Mockito.when(projectRepository.findById(projectId))
        .thenReturn(Optional.of(project));
    Mockito.when(projectMemberRepository.findByProjectIdAndUserId(
        Mockito.eq(projectId), Mockito.eq(1L)))
        .thenReturn(Optional.of(member));
    Mockito.when(testSuiteRepository.save(Mockito.any()))
        .thenReturn(saved);

    // Request avec les test data générés
    CreateTestSuiteRequest request = new CreateTestSuiteRequest();
    request.setName("TestProject");
    request.setType("UNIT");
    request.setGitRepoUrl("https://github.com/testproject.git");
    request.setGitBranch("master");
    request.setModulePath("/src/main/java/com/example/test");

    // Act
    TestSuiteResponse response = testSuiteService.add(projectId, request);

    // Assert
    Assert.assertNotNull(response);
    Assert.assertEquals(response.getName(), "TestProject");
    Mockito.verify(testSuiteRepository, Mockito.times(1)).save(Mockito.any());
    Mockito.verify(projectRepository, Mockito.times(1)).findById(projectId);
}
```

project create

```
package suites.unit; //

import com.pfe.platform.ms_gestion.service.ProjectService;
import com.pfe.platform.ms_gestion.repository.ProjectRepository;
import com.pfe.platform.ms_gestion.repository.ProjectMemberRepository;
import com.pfe.platform.ms_gestion.entity.Project;
import com.pfe.platform.ms_gestion.dto.request.CreateProjectRequest;
import com.pfe.platform.ms_gestion.dto.response.ProjectResponse;
import org.mockito.*;
import org.testng.annotations.*;
import static org.testng.Assert.*;
import static org.mockito.Mockito.*;

public class ProjectServiceTest {

    private AutoCloseable mocks; //

    @Mock private ProjectRepository projectRepository;
    @Mock private ProjectMemberRepository projectMemberRepository;
    @InjectMocks private ProjectService sut;

    @BeforeMethod
    public void setUp() throws Exception {
        mocks = MockitoAnnotations.openMocks(this); //
    }

    @AfterMethod
    public void tearDown() throws Exception {
        mocks.close(); //
    }

    @Test
    public void test_create_happy_path() {
        // given
        CreateProjectRequest request = new CreateProjectRequest();
        request.setName("test");

        Project saved = new Project();
        saved.setId(1L);
        saved.setName("test"); // entité réelle, pas null
        when(projectRepository.save(any())).thenReturn(saved);

        // when
        ProjectResponse result = sut.create(request);

        // then
        assertNotNull(result);
        assertEquals(result.getName(), "test");
        verify(projectRepository, times(1)).save(any());
    }
}
```